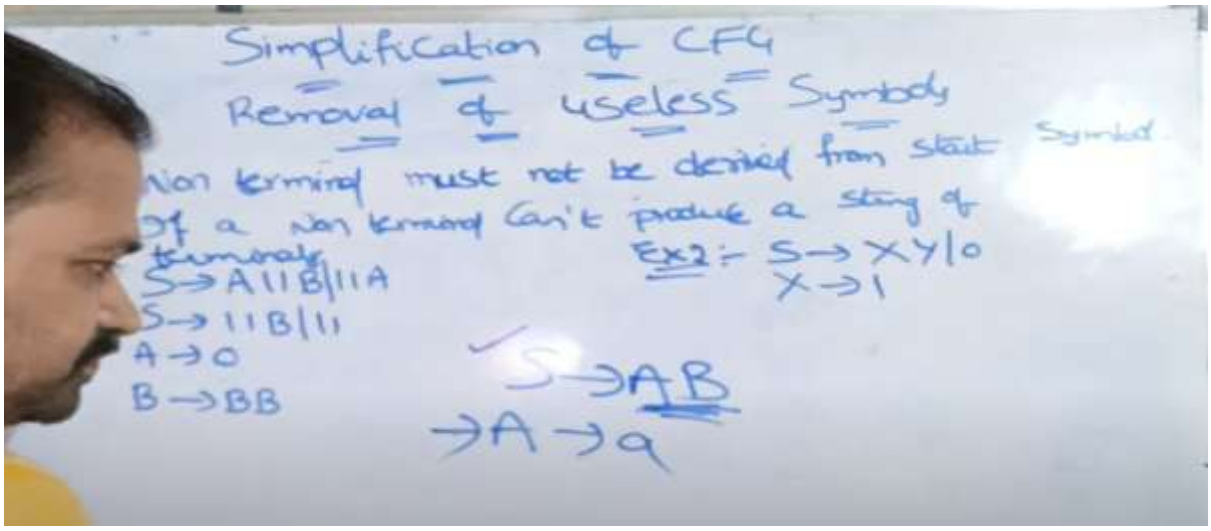
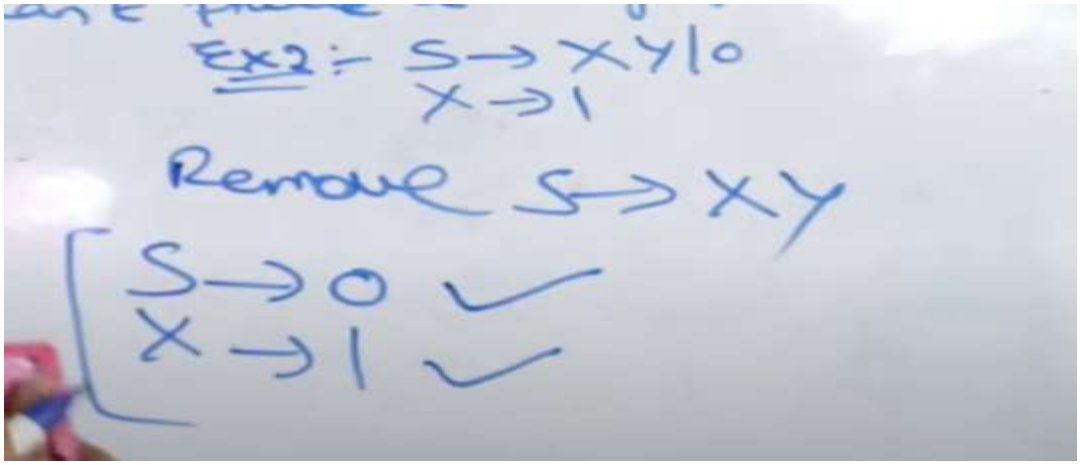
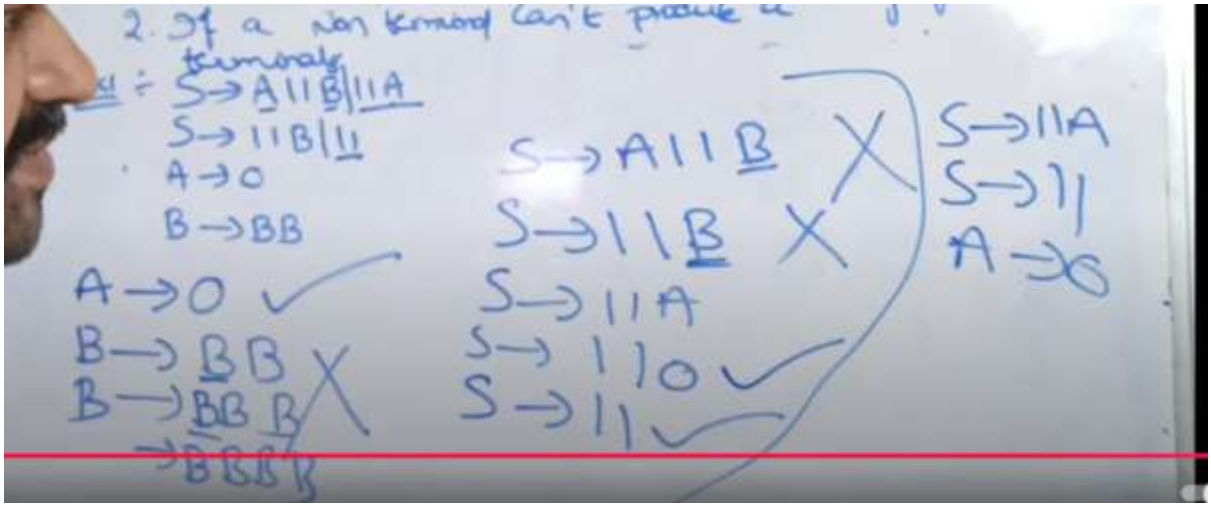
UNIT – IV

Normal Forms for Context- Free Grammars:

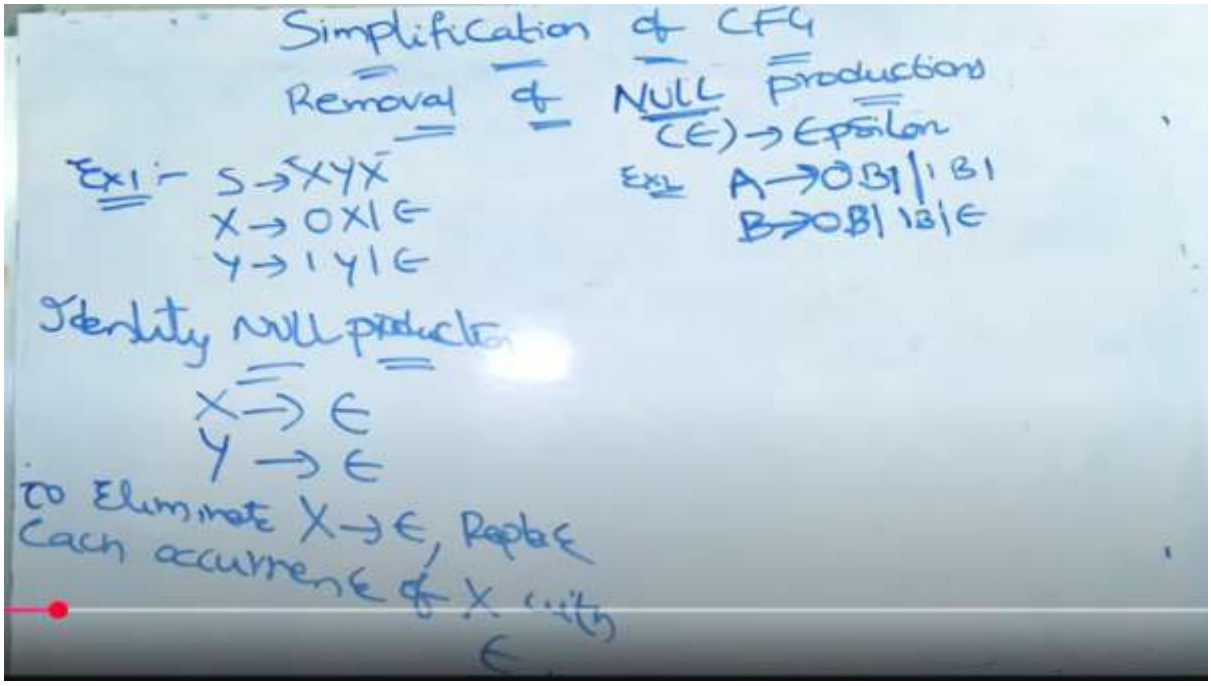
1.Removing useless symbols

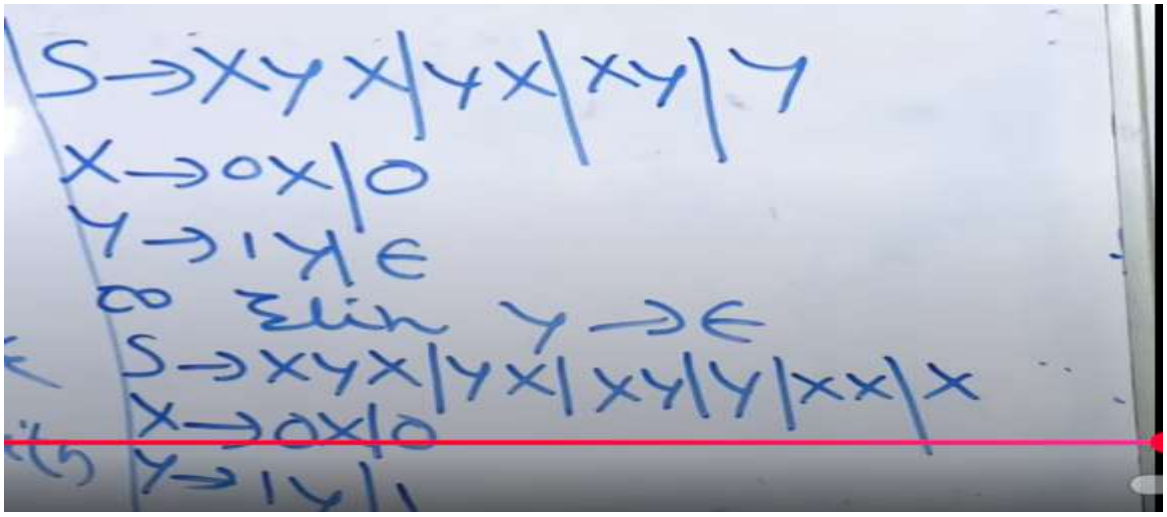


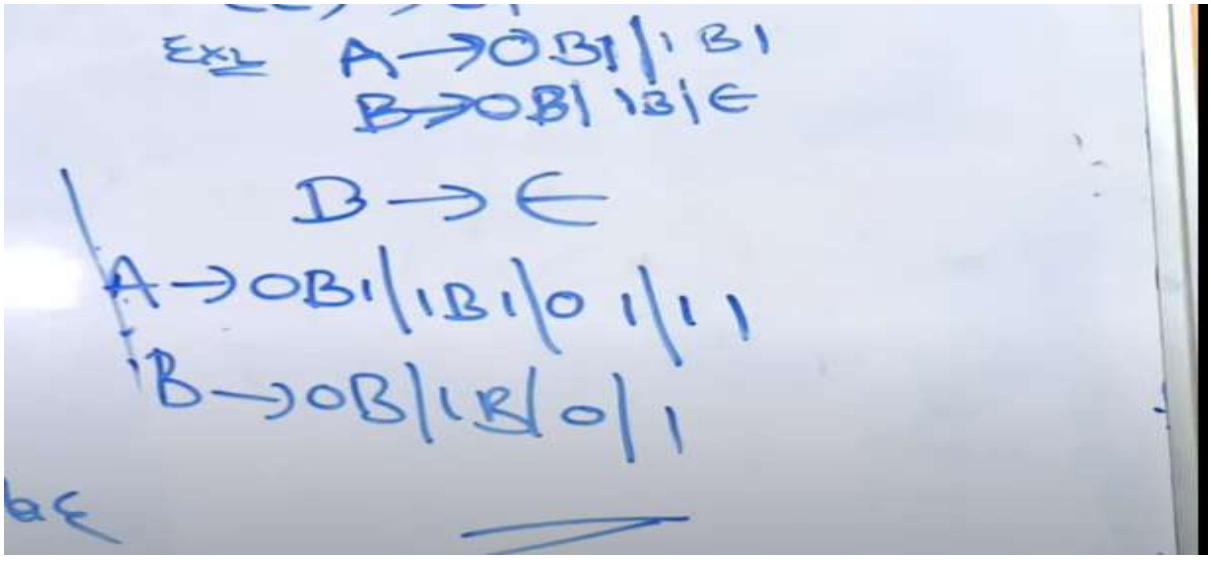




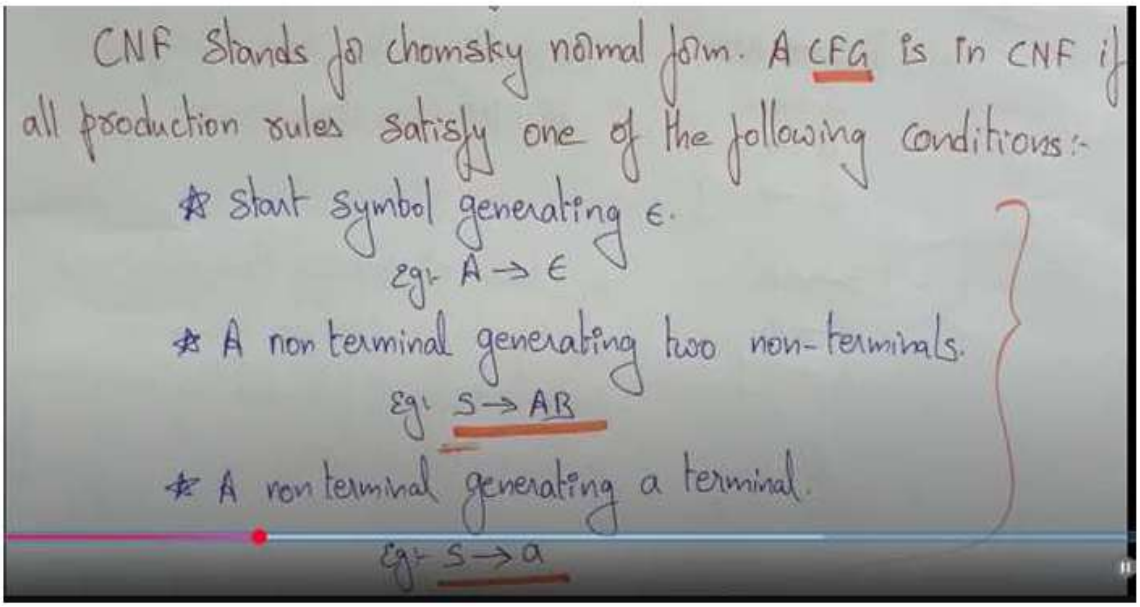
2. Eliminating €- Productions.

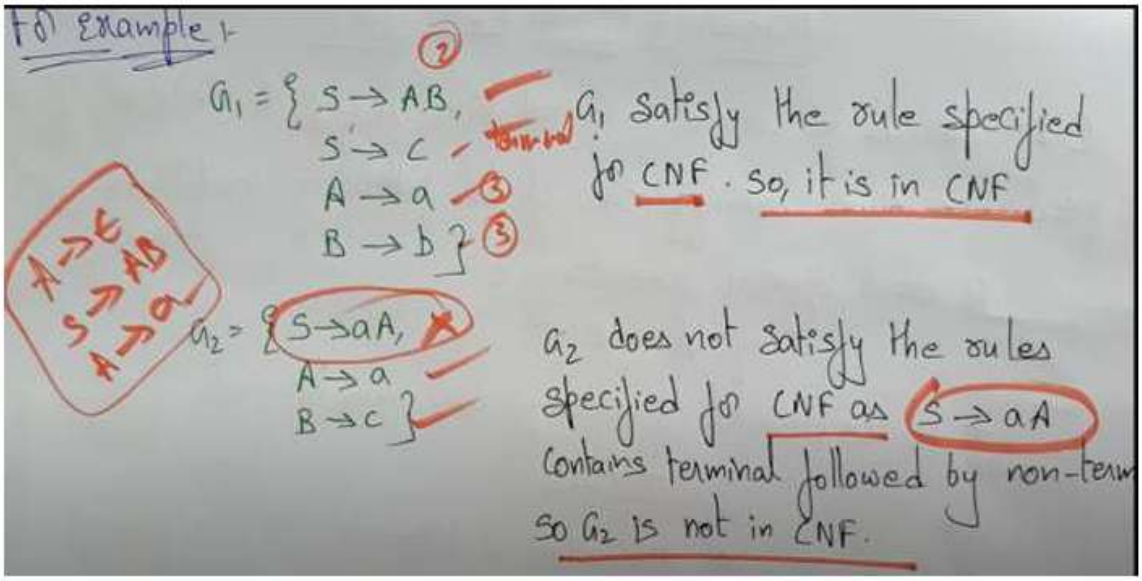




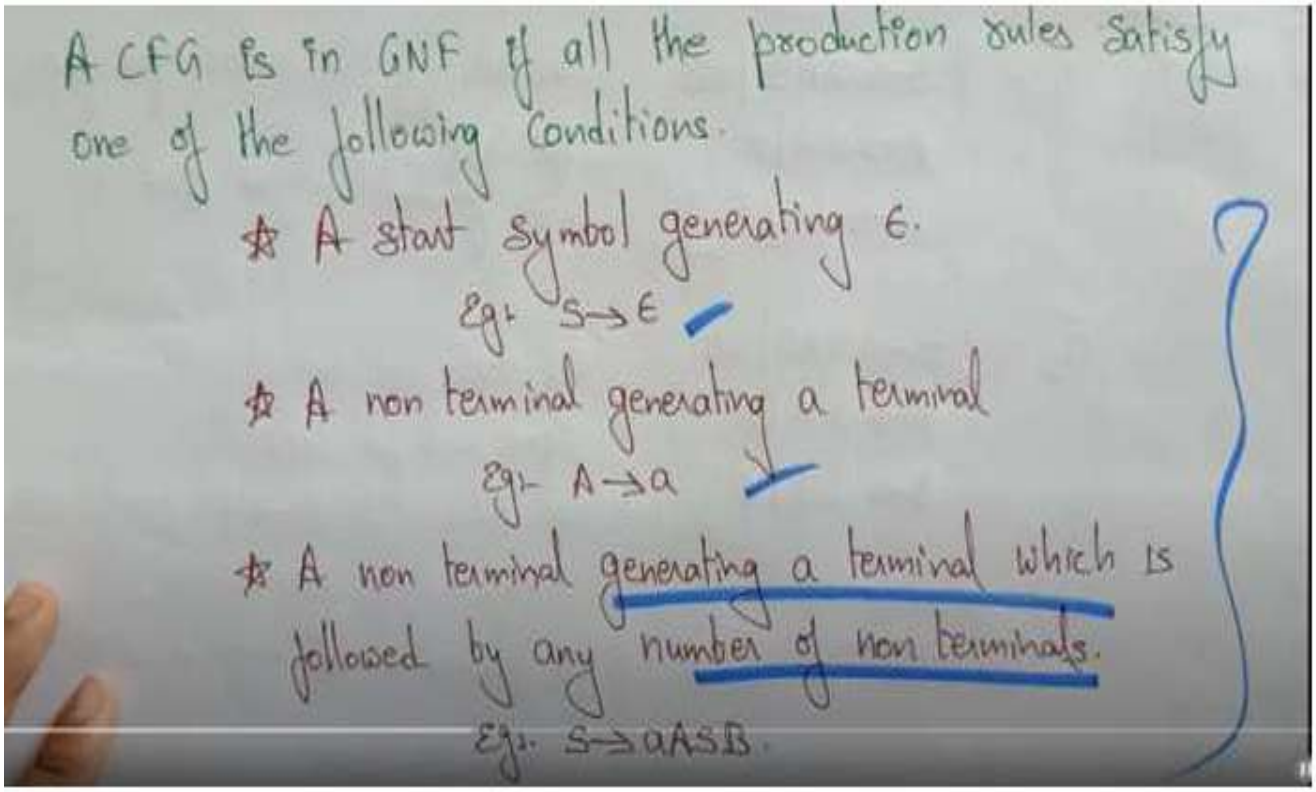


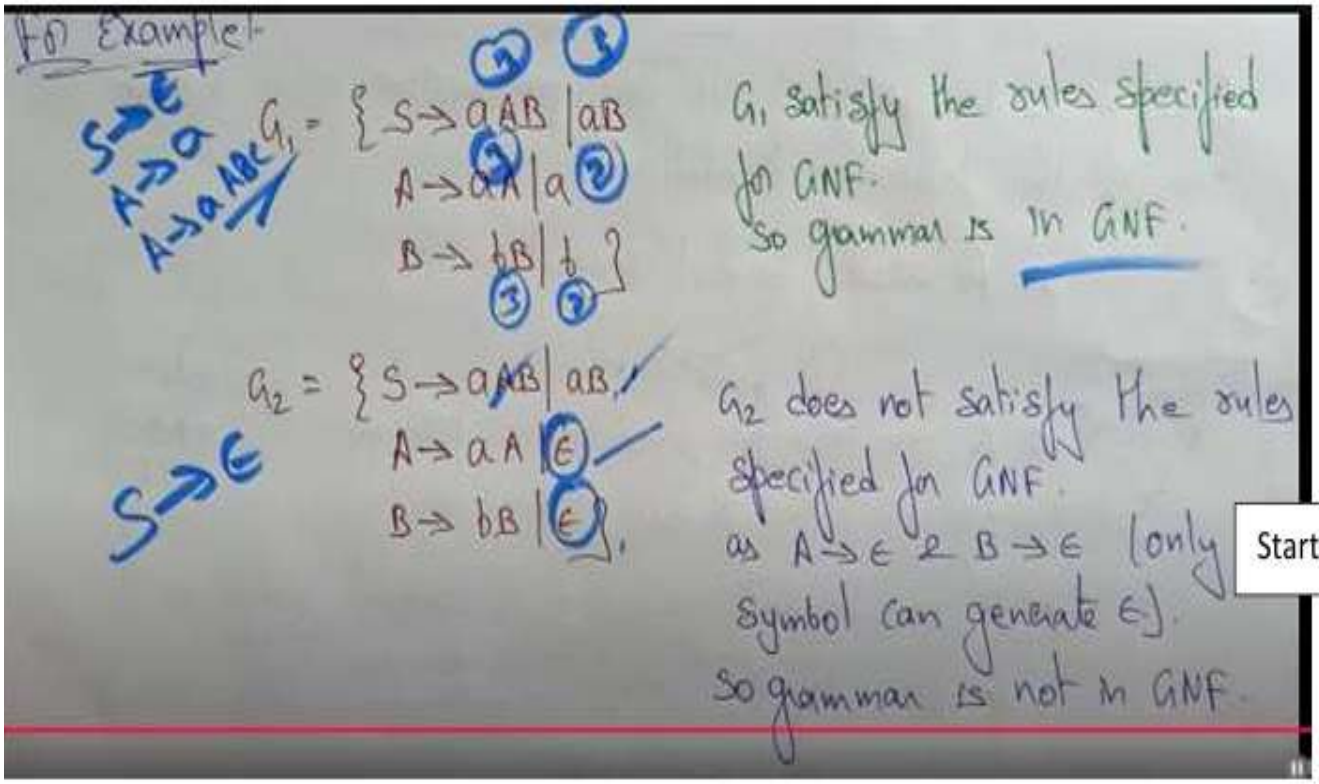
3. Chomsky Normal form (CNF)



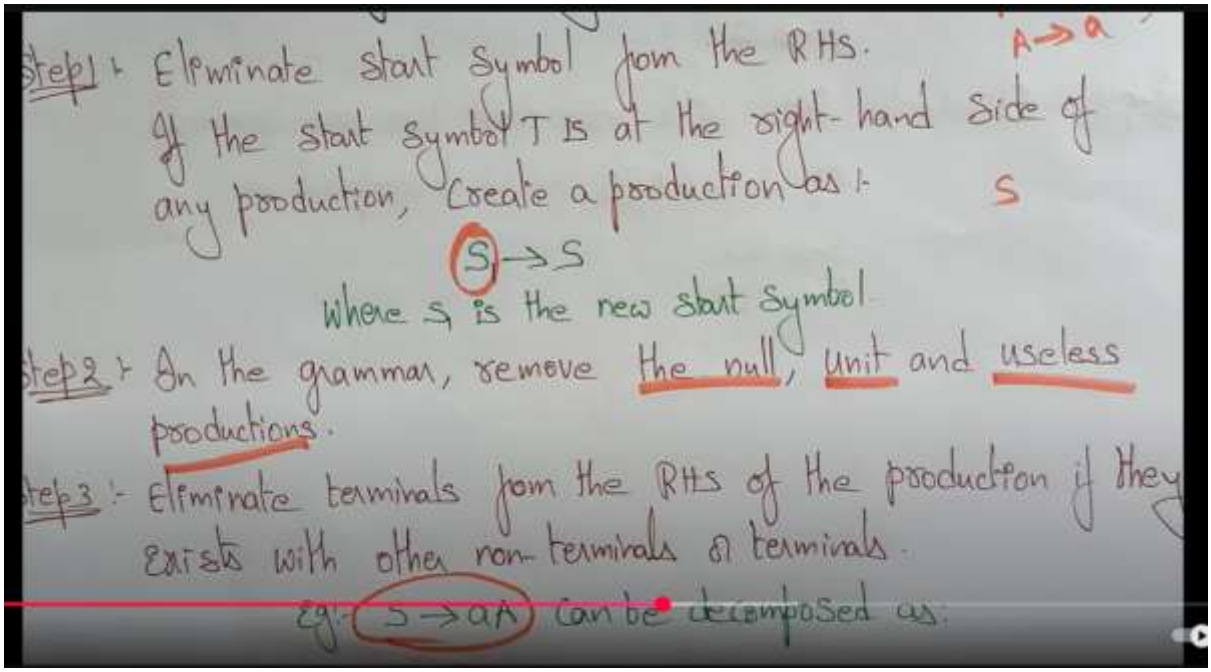


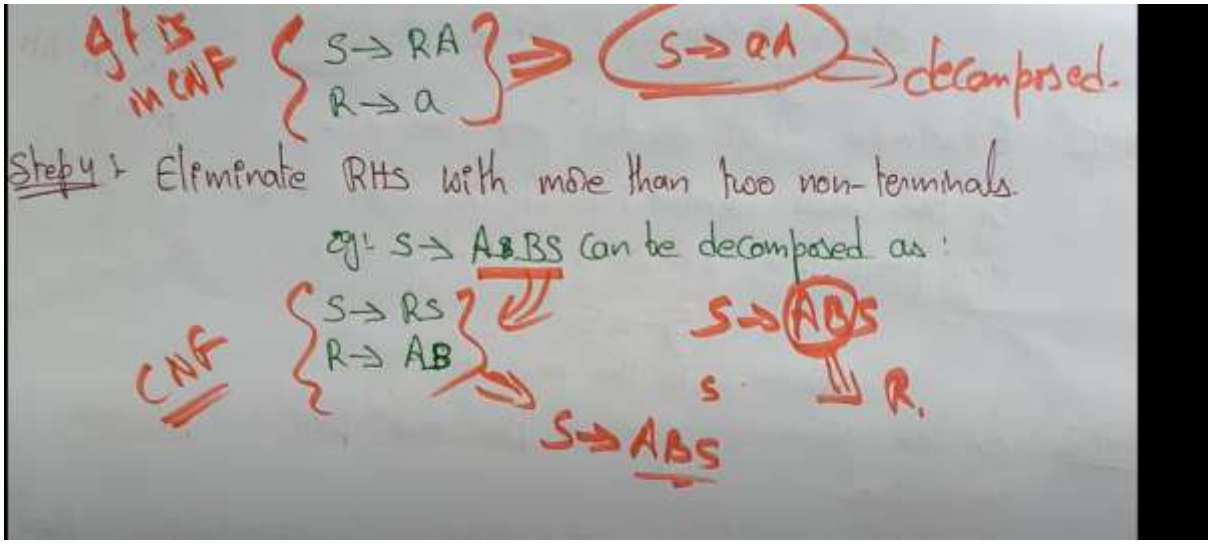
4.Greinbach Normal form(GNF)





Steps to convert CFG to CNF





Example: Convert the given CFG to CNF.

Consider the given grammar G_1 as

$S \to a \mid aA \mid B$

$A \to aBB \mid \epsilon$

$B \to Aa \mid b$

rules CN

$S \to \epsilon$

$A \to AB$

$A \to a$

Solution:

Step 1:

$S_1 \to S$ // new production.

$S \to a \mid aA \mid B$

$A \to aBB \mid \epsilon$

$B \to Aa \mid b$

Step 2:-

$A \to \epsilon$ null production, we have to remove ϵ.

$S_1 \to S$ unit production.

$S \to a \mid aA \mid B$

$A \to aBB$

$B \to Aa \mid b \mid a$

$A \to \epsilon$

replace A with & write that

Simplify CFG

To eliminate unit production, replace B

with B production rule.

$S_1 \to a \mid aA \mid Aa \mid b$

$S \to a \mid aA \mid Aa \mid b$

$A \to aBB$

$B \to Aa \mid b \mid a$

Simplify CFG

rew

rew

rew

Step 3:

$S_0 \to aA/Aa$

$S \to aA/Aa$

$A \to aBB$

$B \to Aa$

$\Rightarrow \quad X \to a \quad \Rightarrow$

$S_0 \to a/XA/AX/b$

$S \to a/XA/AX/b$

$A \to XBB$

$B \to AX/b/a$

$X \to a$

$S_0 \to a/XA/AX/b$

$S \to a/XA/AX/b$

$A \to RB$

$R \to XB$ $\Rightarrow A \to XBB$

$B \to AX/b/a$

$X \to a$

$A \to XBB$ decomposed into $A \to XR$, $R \to BB$

$\therefore$ The given grammar in CNF

5.Pumping Lemma for Context-Free Languages:

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{ V, \Sigma, S, P \}$ where

V = Set of Variables or Non-Terminal Symbols

Σ = Set of Terminal Symbols

S = Start Symbol

P = Production Rule

Context Free Grammar has Production Rule of the form

$A \rightarrow a$

where, $a = \{V \cup \Sigma\}^*$ and $A \in V$

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

If A is a Context Free Language, then, A has a Pumping Length 'P' such that any string 'S', where $|S| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

(1) $u v^i x y^i z$ is in A for every $i \geq 0$

(2) $|v y| > 0$

(3) $|v x y| \leq P$

To prove that a Language is Not Context Free using Pumping Lemma (for CFL) follow the steps given below: (We prove using CONTRADICTION)

-> Assume that A is Context Free

-> It has to have a Pumping Length (say P)

-> All strings longer than P can be pumped $|S| \geq P$

-> Now find a string 'S' in A such that $|S| \geq P$

-> Divide S into uvxyz

-> Show that $u v^i x y^i z \notin A$ for some i

-> Then consider the ways that S can be divided into uvxyz

-> Show that none of these can satisfy all the 3 pumping conditions at the same time

-> S cannot be pumped == CONTRADICTION

Show that $L = \{ a^N b^N c^N \mid N \geq 0 \}$ is Not Context Free

-> Assume that L is Context Free

-> L must have a pumping length (say P)

-> Now we take a string S such that $S = a^P b^P c^P$

-> We divide S into parts u v x y z

Eg. P = 4 So, $S = a^4 b^4 c^4$

Case I : v and y each contain only one type of symbol

a a a a b b b b c c c c

u v x y z

$uv^i xy^i z \quad (i = 2)$

$uv^2 xy^2 z$

a a a a a a b b b b c c c c c

$a^6 b^4 c^5 \notin L$

Case II : Either v or y has more than one kind of symbols

a a a a b b b b c c c c

u v x y z

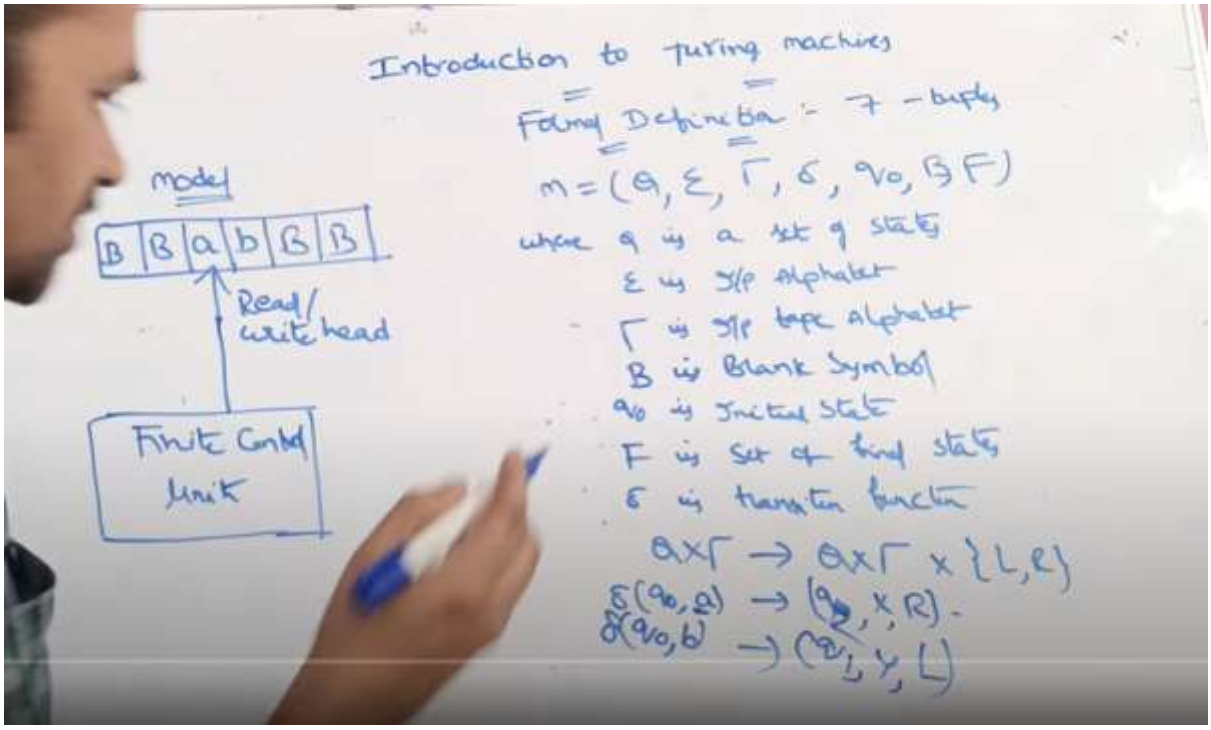
$uv^i xy^i z \quad (i = 2)$

$uv^2 xy^2 z$

$a^N b^N c^N$

a a a a b b a a b b b b c c c c $\notin L$

6.Turing Machines:

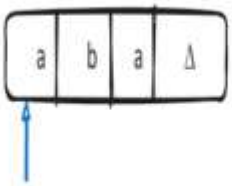


Language accecpted by turing machine

Example:

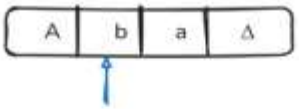
- Construct a Turing Machine which accepts the language of aba Over Σ={a,b}

Initially



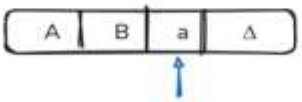
Move 1:

$\delta(q0,a) = \delta(q1,A,R)$



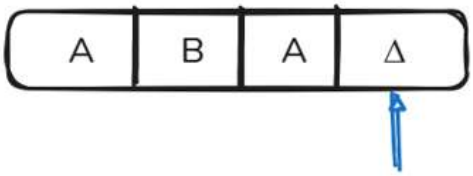
Move 2:

$\delta(q1,b) = \delta(q2,B,R)$



Move 3:

$\delta(q2,a) = \delta(q3,A,R)$



Move 4:

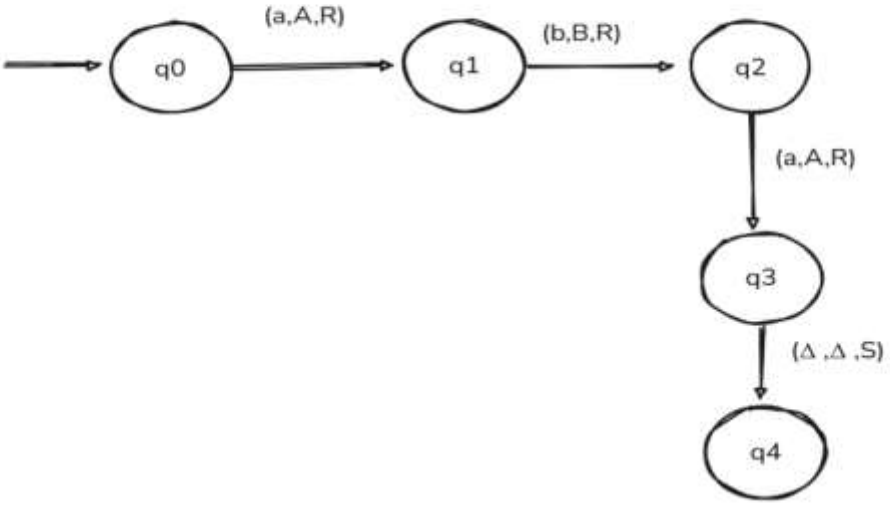
$\delta(q3, \Delta) = \delta(q4, \Delta)$

Here q4 is the Halt

Transition Table

	a	b	Δ
q0	(q1,A,R)	-	-
q1	-	(q2,B,R)	-
q2	(q3,A,R)	-	-
q3	-	-	(q4, Δ,S)
q4	-	-	-

Transition Diagram



ID of a TM

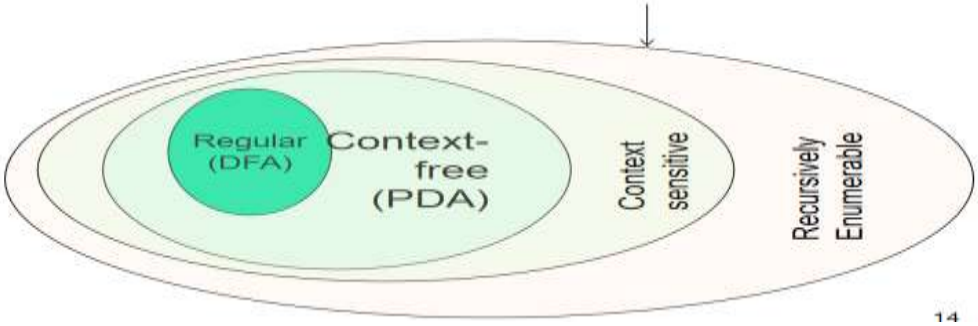
- Instantaneous Description or ID :
  - $X_1X_2\ldots X_{i-1}qX_iX_{i+1}\ldots X_n$
    means:
    - q is the current state
    - Tape head is pointing to X_i
    - $X_1X_2\ldots X_{i-1}X_iX_{i+1}\ldots X_n$ are the current tape symbols

- $\delta(q,X_i) = (p,Y,R)$ is same as:
  $X_1\ldots X_{i-1}qX_i\ldots X_n$ |---- $X_1\ldots X_{i-1}YpX_{i+1}\ldots X_n$
- $\delta(q,X_i) = (p,Y,L)$ is same as:
  $X_1\ldots X_{i-1}qX_i\ldots X_n$ |---- $X_1\ldots pX_{i-1}YX_{i+1}\ldots X_n$

7.The language of turng machine

Language of the Turing Machines

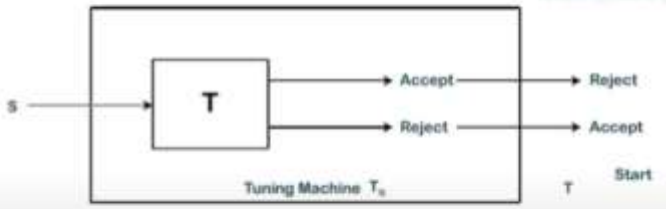
- *Recursive Enumerable (RE) language*



14

Recursively Enumerable Languages

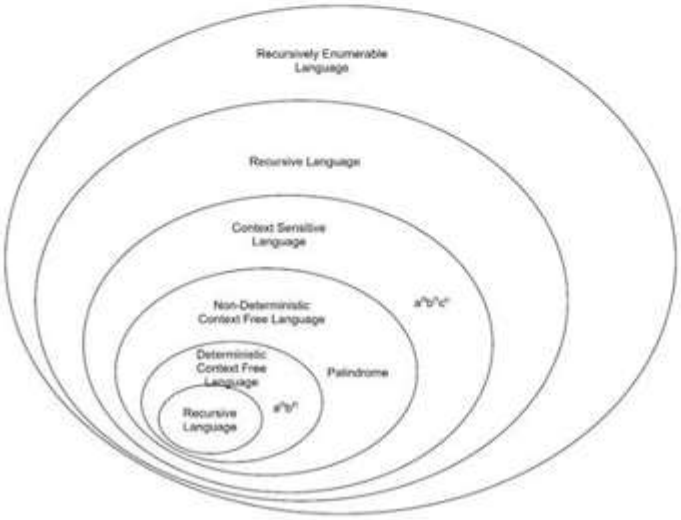
- Recursively enumerable languages are those for which a Turing machine can enumerate all valid strings.
- Every recursively enumerable language can be recognized by a Turing machine, but not necessarily decided.
- These languages can be infinite and do not require that the Turing machine halts on all inputs.



Recursively Enumerable Languages

In simple words, a "language" is a collection of strings, like words in a dictionary. A recursively enumerable language is a language where we can create a computer program (or a Turing machine) that can systematically list out all the strings that belong to the language.

Consider a machine that can generate all the possible sentences in the English language, one by one. This machine wouldn't necessarily know which sentences are not in the English language, but it could list out all the valid sentences. This is the idea of a recursively enumerable language. It can enumerate all the strings that are part of the language.



.

Recursive Languages: A Subset of RE Languages

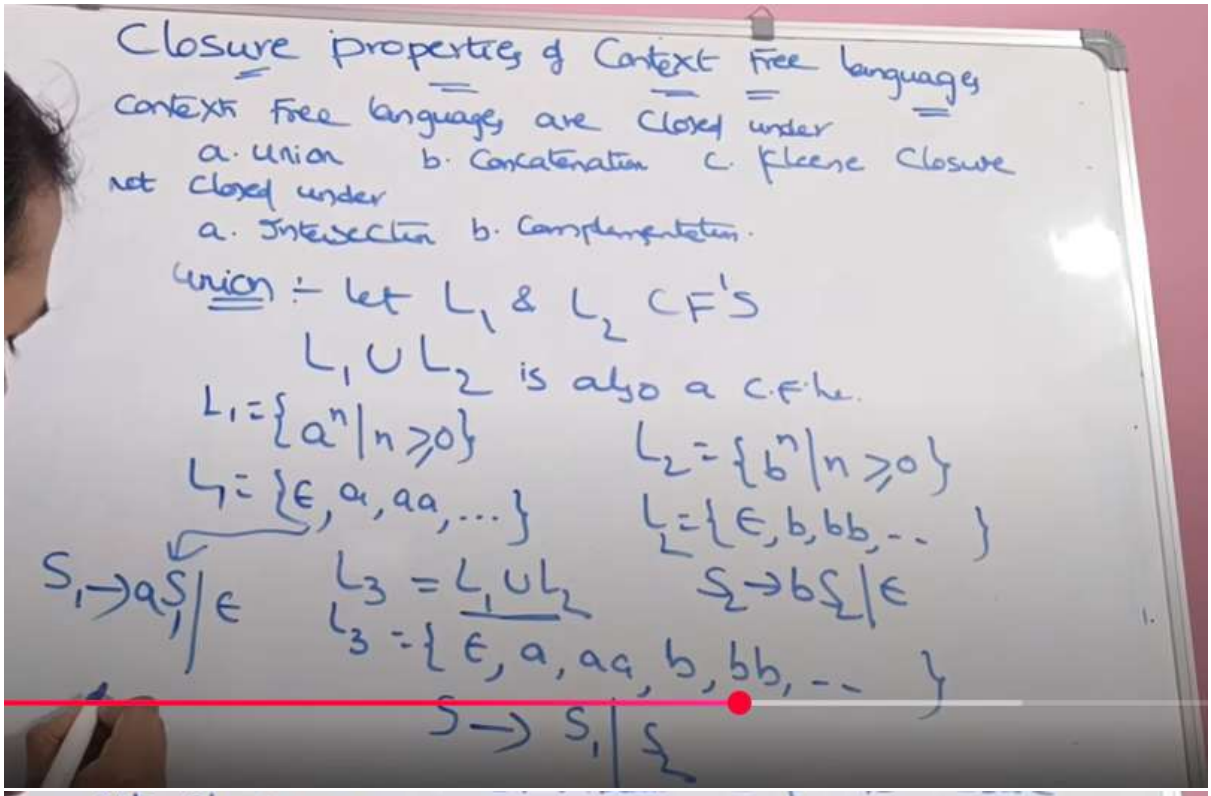
Another important subset of RE languages is recursive language. In a recursive language, the Turing machine not only accepts strings belonging to the language but also always halts for strings that are not in the language

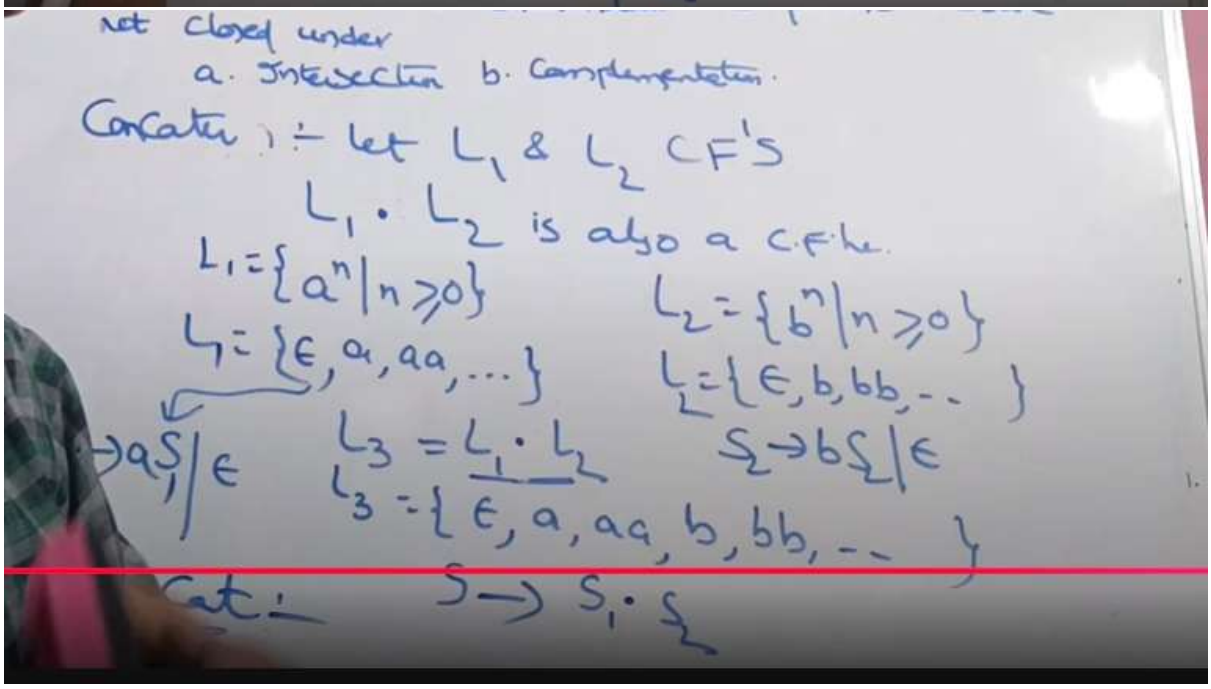
As an example, Consider the language $L=\{a^n b^n c^n | n \geq 0\}$. This language consists of strings where the number of a's, b's, and c's are equal

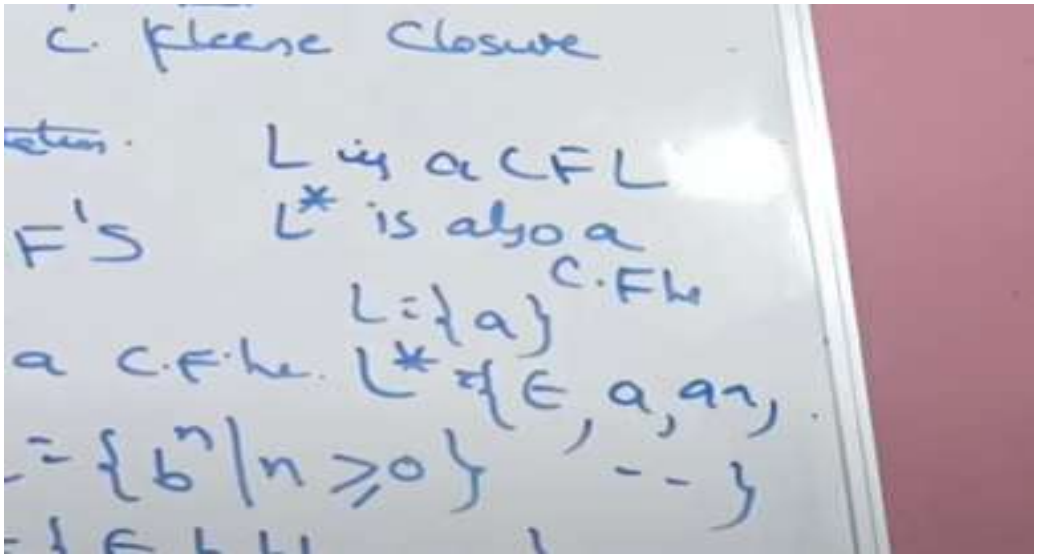
---

- **RE Language** – We can build a Turing machine that starts at the beginning of the string and systematically checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. However, if the string is not of this form, the machine may never halt, potentially looping forever. This makes L a recursively enumerable language.
- **Recursive Language** – We can also construct a Turing machine that checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. If they are not, it reaches a "reject" state and halts. This makes L a recursive language as well.

8.Closure property of CFL







Intersection

L1 ^ L2 doest produce CFL

Complement

L1 ---L2 doest produce CFL

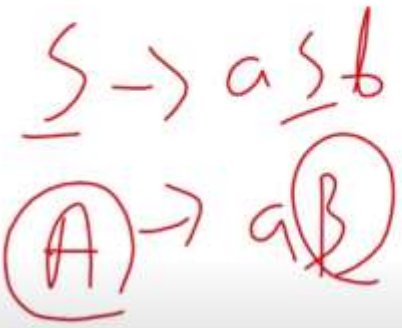
9.Decision Property of CFL

Decision Properties of CFG:

- CFG is decidable for Emptiness , Finiteness, Membership, Equality of CFG of DCFL.
- CFG is undecidable for ambiguity , equality, Regularity of CFG.
- Emptiness:
- Convert the grammar into reduced CFG.
- If the reduced CFG generate at least one string then the Grammar generate non empty language else generate empty language.
- Ex 1:
  - S→aAB/∈
  - A →Bb/AB/a
  - B →BA
  - S→ ∈
  - A→a
  - S→ ∈
  - Non Empty language as|∈|=1
- Ex 2:
  - S→AaB/Aa
  - A →bA/ ∈
  - S→Aa/a
  - A→bA/b
  - Non Empty language as RE(S)=b*a
- Ex 3:
  - ~~S→aAB~~
  - ~~A →bA/a~~
  - Empty language

Finiteness:

If one Grammar if Recursive then Infinite Language .If non recursive then finite language.



Membership:

Membership is a property to verify the string is generated by a Grammar or not.

1.Try to derive the string from Grammar .If it can generate then the string is member of that Grammar

2.Convert the Grammar into CNF . Apply CYK(Cocke–Younger–Kasami) algorithm

CYK uses dynamic programming

Ex :S →AB

A →BA/SA/b

B →BB/BS/a

w=ab ✗

a	b
B	A
A	

w=ba ✓

b	a
A	B
S	

w=aba ✓

a	b	a
B	A	B
A	S	
B,S		

w=abab ✗

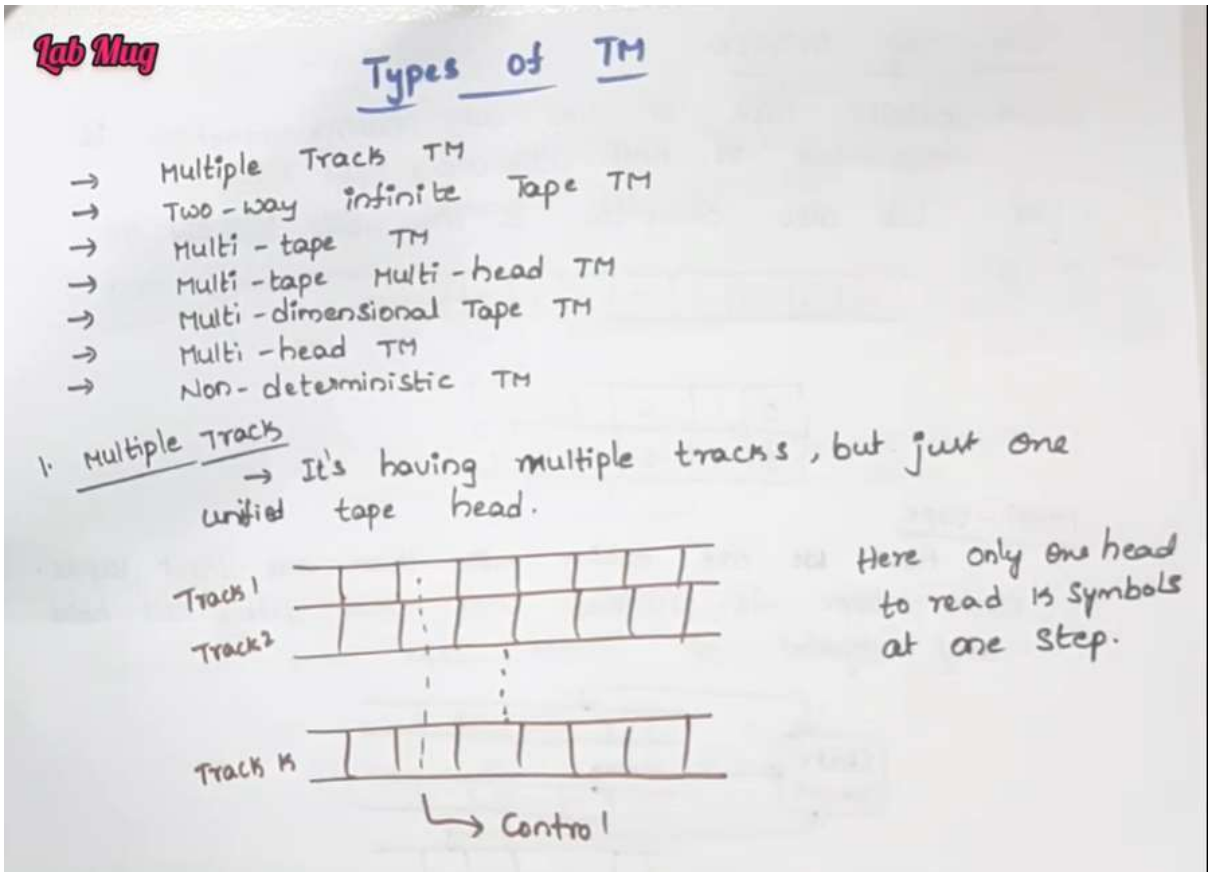
a	b	a	b
B	A	B	A
A	S	A	
B,S	A		
A			

w=abaa ✓

a	b	a	a
B	A	B	B
A	S	B	
B,S	S		
B,S			

UNIT-5

1.Types of Turing machine



Lab Mug

Two way infinite

→ Infinite tape of two-way turing machine is unbounded in both directions (left & right)

→ It's also converted to one-way infinite TM

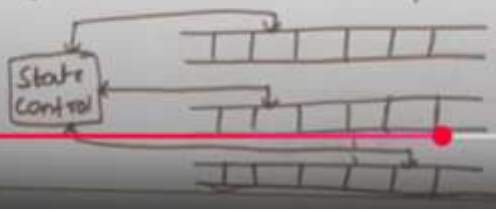
Ex:

...	-4	-3	-2	-1	0	1	2	3	4	- -

0	1	2	3	4
$	-1	-2	-3	-4

Multi-tape

Here we are taking more than one input tapes. Each tape is divided into cells. Every cell hold any symbol of finite tape



Lab Mug

Multi-tape Multi head TM

→ It has multiple tapes and multiple heads

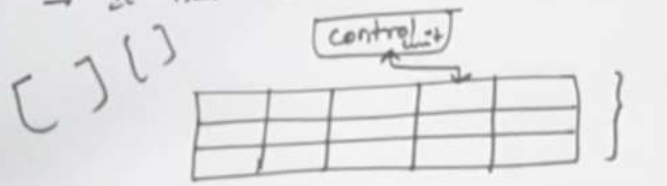
→ Each tape is controlled by a separate head

→ It's also converted to standard TM

Multi-dimensional Tape

→ It has a multi-dimensional tape.

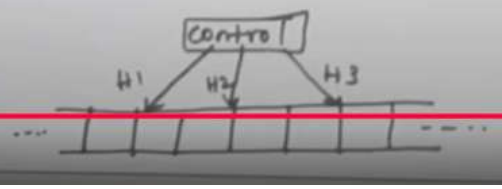
→ It can move left, right, up and down

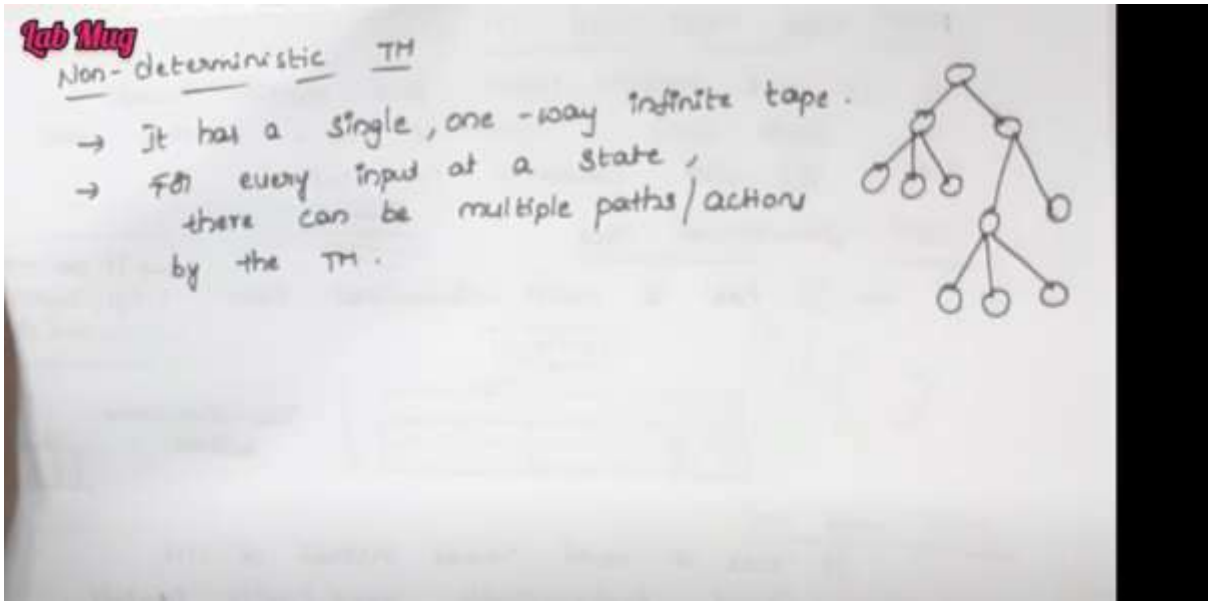


Multi-head TM

It has a no. of heads instead of one

→ Every head independently read/write symbols





2.decidibility and undecidebility

8.5 DECIDABILITY OF PROBLEMS

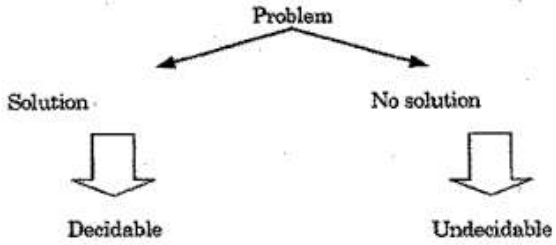
In our general life, we have several problems and some of these have solution also, but some have not. Simply, we say a problem is decidable if there is a solution otherwise undecidable.

Example : consider following problems and their possible answers.

1. Does the sun rise in the east ? (YES)
2. Does the earth move around the sun ? (YES)
3. What is your name ? (FLAT)
4. Will tomorrow be a rainy day ? (No answer)

We have solutions (answers) for all problems except the last. We can not answer the last problem, because we have no way to tell about the weather of tomorrow, but to some extent we can only predict. So, the last problem is undecidable and remaining problems are decidable.

So, if a problem can be solved or answered based on some algorithm then it is decidable otherwise undecidable.



Deciadability & Undecidability in TOC

(a) Decidable languages & Undecidable langs

→ A problem is said to be decidable if we can always construct an alg. to solve the problem

→ A language is decidable if there is a Turing Machine that accepts the language & halts on every I/P string with a solution Yes/No.

→ Decidable problems are also known as Turing Decidable problems because a Turing machine always halts on Every I/P by accepting & rejecting.

Deciadability & Undecidability in TOC

(a) Decidable languages & Undecidable langs

→ Here the language is not decidable.

→ One corresponding problem is unsolvable.

→ There is no Algorithm/Turing Machine that give an answer for all I/P instances of problem.

→ In undecidability problem there will always be a chance that will lead Turing Machine into an infinite loop without providing answer.

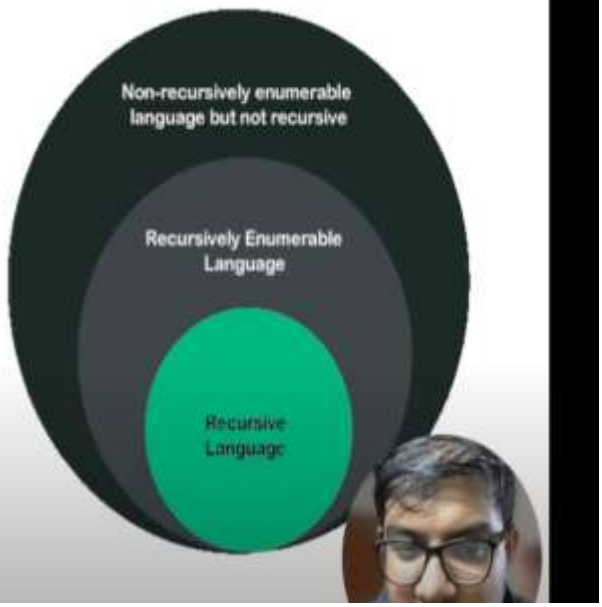
Ex: Find suitable Integers which satisfy Eq. $a^n + b^n = c^n \; \forall \; n \geq 2$

If we give this problem to TM the TM machine might go into infinite loop.

3. A Language that is Not Recursively Enumerable

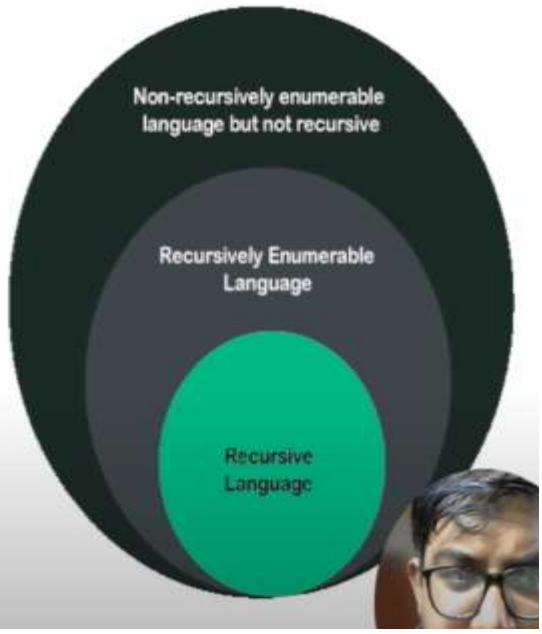
Languages That Are Not Recursively Enumerable

- Some languages cannot even be enumerated by a Turing machine, making them more complex than undecidable languages.
- An example of such a language is the set of all Turing machines that do not halt on empty input.
- These languages are not only undecidable but also fall outside the class of recursively enumerable languages.



Characteristics of Non-Recursively Enumerable Languages

- Non-recursively enumerable languages cannot be recognized by any Turing machine.
- They are often associated with problems that are too complex or too ambiguous for computation.
- Understanding these languages requires delving into deeper areas of logic and set theory.



4.Recursive Language

Recursive Languages: A Subset of RE Languages

Another important subset of RE languages is recursive language. In a recursive language, the Turing machine not only accepts strings belonging to the language but also always halts for strings that are not in the language

As an example, Consider the language $L=\{a^n b^n c^n | n \geq 0\}$. This language consists of strings where the number of a's, b's, and c's are equal

- **RE Language** – We can build a Turing machine that starts at the beginning of the string and systematically checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. However, if the string is not of this form, the machine may never halt, potentially looping forever. This makes L a recursively enumerable language.
- **Recursive Language** – We can also construct a Turing machine that checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. If they are not, it reaches a "reject" state and halts. This makes L a recursive language as well.

Propertied of Recursive language

Closure Properties of Recursive Languages

Recursive languages possess an interesting property called closure. This means that certain operations performed on recursive languages result in another recursive language.

Here are some key closure properties −

Union

If L1 and L2 are two recursive languages, their union (L1 ∪ L2) is also recursive. Imagine a machine that checks if a string belongs to L1 or L2; if it does, it accepts. Since both machines for L1 and L2 will eventually halt, this combined machine will also halt, making the union recursive.

Concatenation

If L1 and L2 are two recursive languages, their concatenation (L1.L2) is also recursive. Imagine a machine that first checks if the first part of the string belongs to L1, and if it does, it checks the remaining part of the string for L2. Since both L1 and L2 machines halt, this combined machine will also halt.

Kleen Closure

If L1 is a recursive language, its Kleen closure (L1*) is also recursive. This means the language including all possible combinations of strings from L1 concatenated together, including the empty string, is also recursive.

Intersection

If L1 and L2 are two recursive languages, their intersection (L1 ∩ L2) is also recursive. Imagine a machine that checks if a string belongs to both L1 and L2. Since both L1 and L2 machines halt, this combined machine will also halt.

Complement

If L1 is a recursive language, its complement (L1') is also recursive. This means the language containing all strings "not" in L1 is also recursive.

5.Undecidebility problem of turing machine

Undecidable Problems about Turing Machines

In this section, we will first discuss about halting problem in general and then about TM.

Halting Problem (HP)

The **halting problem** is a decision problem which is informally stated as follows :

"Given a description of an algorithm and a description of its initial arguments, determine whether the algorithm, when executed with these arguments, ever halts. The alternative is that a given algorithm runs forever without halting."

Alan Turing proved in 1936 that there is no general method or algorithm which can solve the halting problem for all possible inputs. An algorithm may contain loops which may be infinite or finite in length depending on the input and behaviour of the algorithm . The amount of work done in an algorithm usually depends on the input size. Algorithms may consist of various number of loops, nested or in sequence. The HP asks the question :

Given a program and an input to the program, determine if the program will eventually stop when it is given that input ?

One thing we can do here to find the solution of HP. Let the program run with the given input and if the program stops and we conclude that problem is solved. But, if the program doesn't stop in a reasonable amount of time, we can not conclude that it won't stop. The question is : " how long we can wait .... ?" . The waiting time may be long enough to exhaust whole life. So, we can not take it as easier as it seems to be. We want specific answer, either "YES" or "NO", and hence some algorithm to decide the answer.

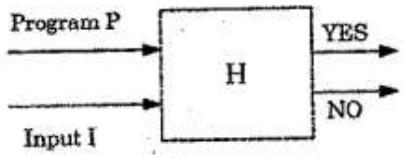
The importance of the halting problem lies in the fact that it is the first problem which was proved undecidable. Subsequently, many other such problems have been described.

Theorem : HP is undecidable.

Proof : This proof was devised by Alan Turing in 1936. Initially, we assume that HP is decidable and the algorithm (solution) for HP is H. The halting problem solution H takes two inputs :

1. Description of TM M i. e. program P and
2. Input I for the program P.

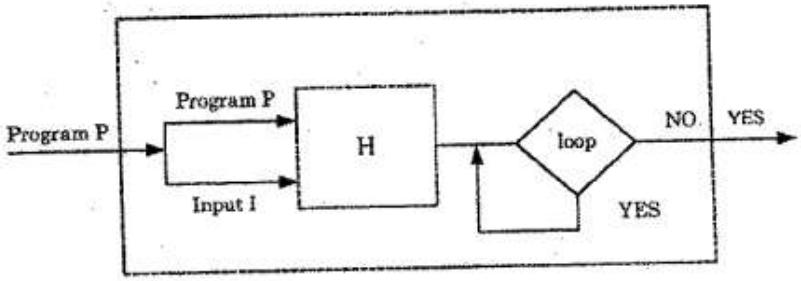
H generates an output "YES" if H determines that P stops on input I or it outputs "NO" if H determines that P loops as shown in figure(a).



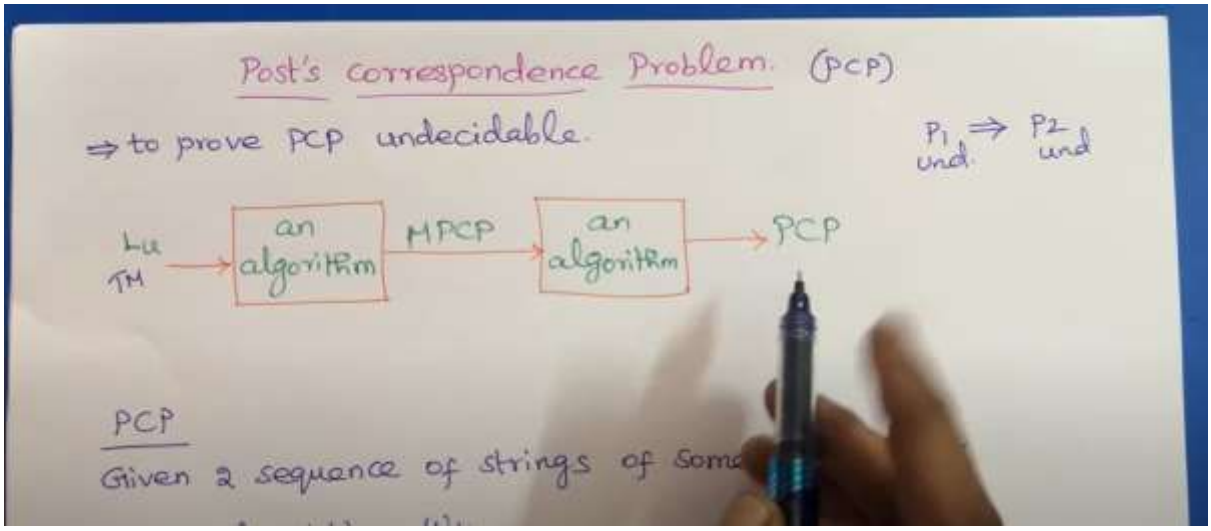
Figure(a)

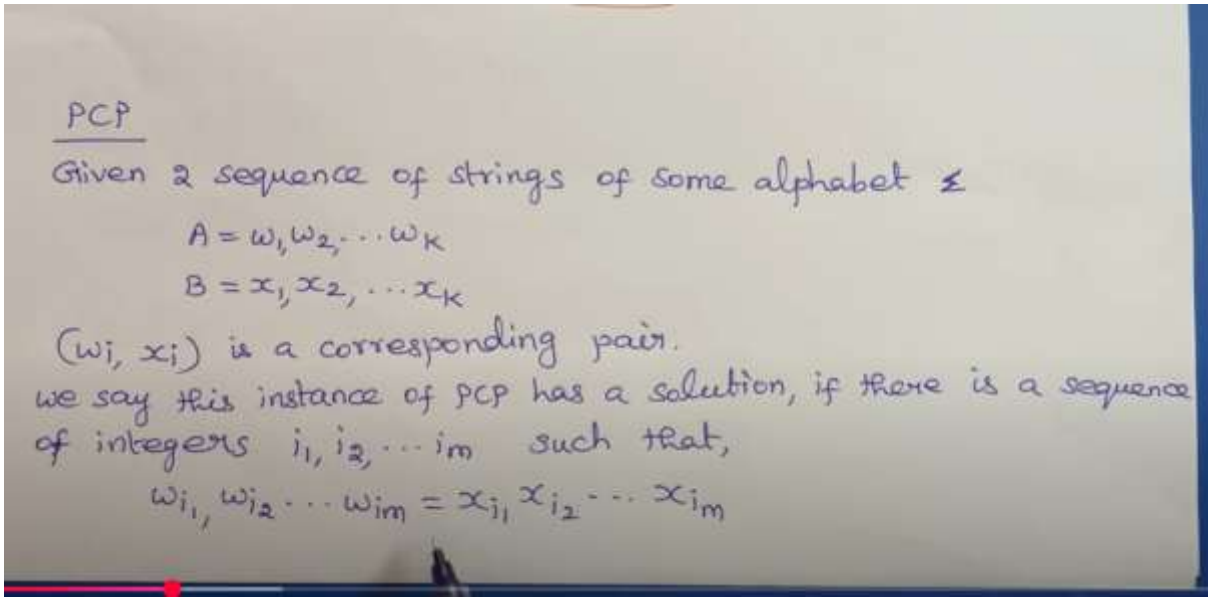
Note : When an algorithm is coded, it is expressed as a string of characters . Input is also coded into the same format. So, after coding, a program and data have no difference in their format of representation and so, a program can be treated a data sometimes and a data can be treated as a program sometimes.

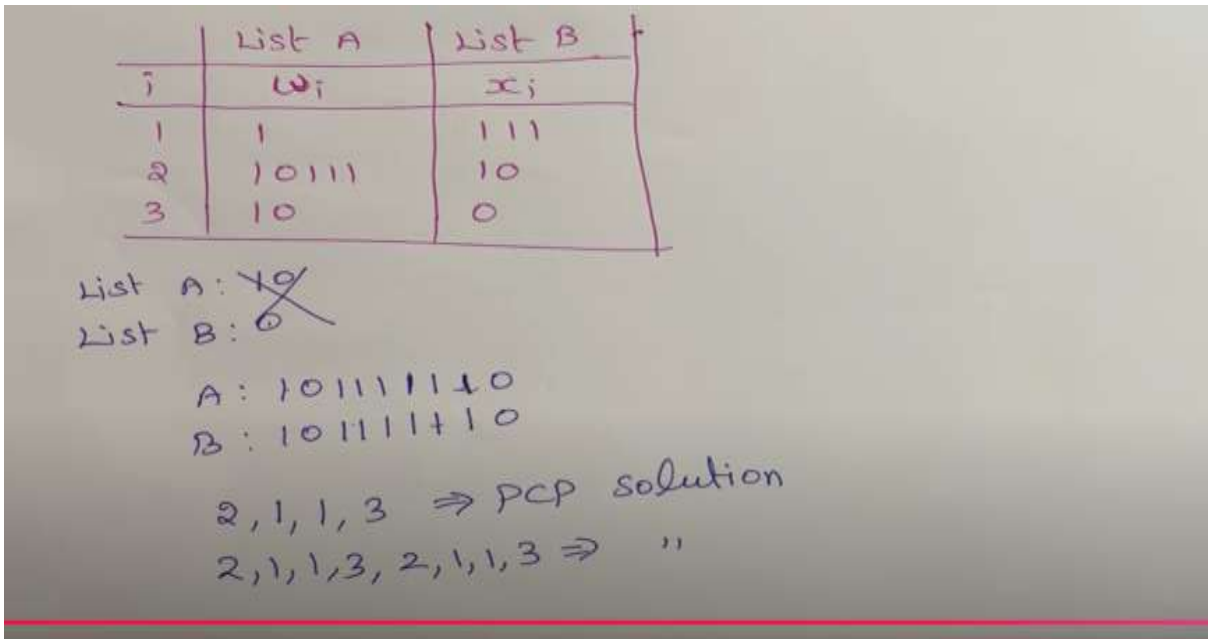
So, now H can be modified to take P as both inputs (the program and its input) and H should be able to determine if P will halt on P as it's input shown in figure(b).

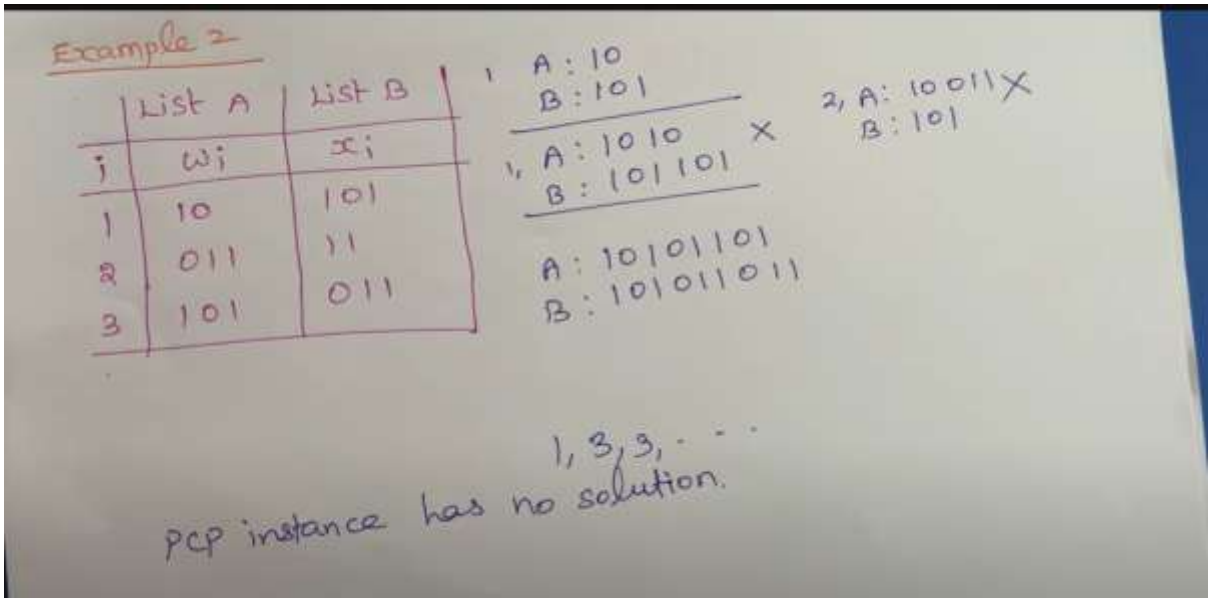


5.Post correspondence problem

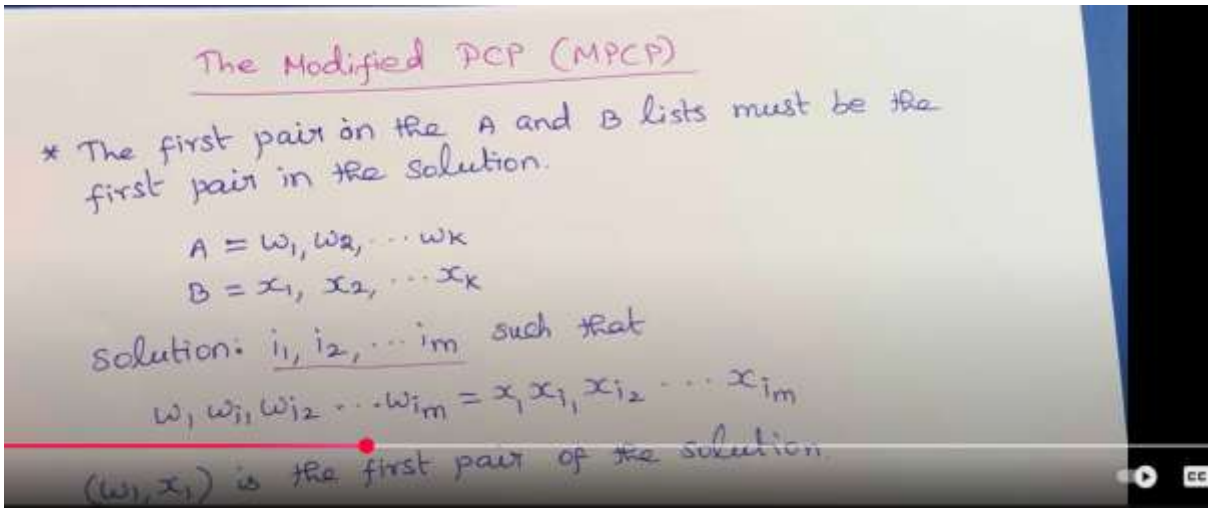


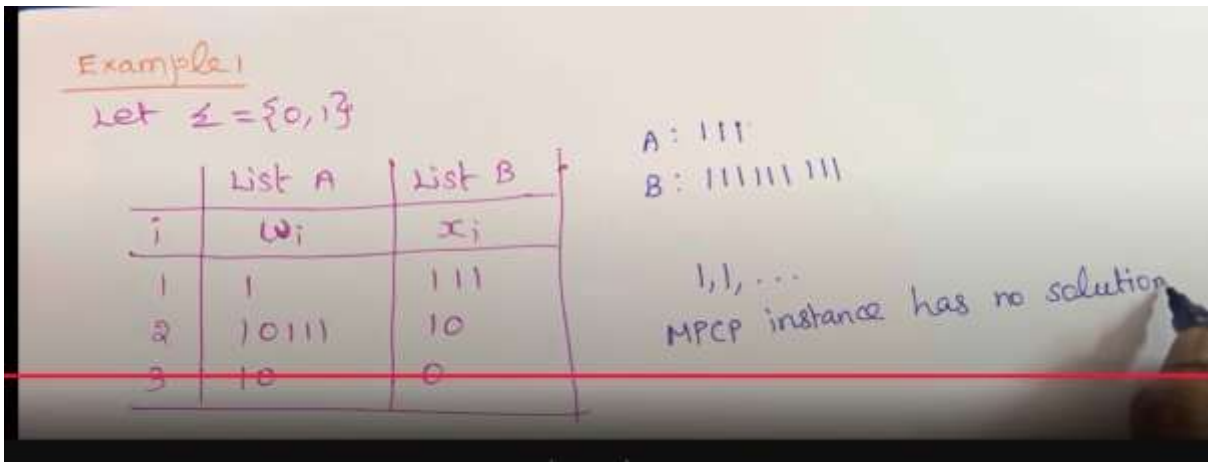






6.Modified pcp





7.7 COUNTER MACHINE

Counter machine has the same structure as the multistack machine, but in place of each stack is a counter. Counters hold any non negative integer, but we can only distinguish between zero and non zero counters.

Counter machines are off - line Turing machines whose storage tapes are semi - infinite, and whose tape alphabets contain only two symbols, Z and B (blank). Furthermore the symbol Z, which serves as a bottom of stack marker, appears initially on the cell scanned by the tape head and may never appear on any other cell. An integer i can be stored by moving the tape head i cells to the right of Z. A stored number can be incremented or decremented by moving the tape head right or left. We can test whether a number is zero by checking whether Z is scanned by the head, but we cannot directly test whether two numbers are equal.

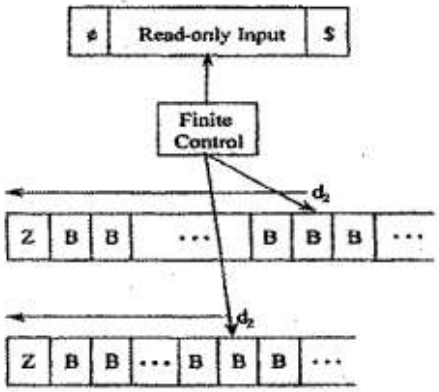


FIGURE : Counter Machine

¢ and $ are customarily used for end markers on the input. Here Z is the non blank symbol on each tape. An instantaneous description of a counter machine can be described by the state, the input tape contents, the position of the input head, and the distance of the storage heads from the symbol Z (shown here as d_1 and d_2). We call these distances the counts on the tapes. The counter machine can only store a count an each tape and tell if that count is zero.

Power of Counter Machines

- Every language accepted by a counter Machine is recursively enumerable.
- Every language accepted by a one - counter machine is a CFL so a one - counter machine is a special case of one - stack machine i. e., a PDA

Unit-3

Context Free Grammer

1.

5.1 CONTEXT FREE GRAMMARS

A grammar $G = (V, T, P, S)$ is said to be a CFG if the productions of G are of the form :

$$A \rightarrow \alpha, \text{ where } \alpha \in (V \cup T)^*$$

The right hand side of a CFG is not restricted and it may be null or a combination of variables and terminals. The possible length of right hand sentential form ranges from 0 to ∞ i.e., $0 \leq |\alpha| \leq \infty$.

As we know that a CFG has no context neither left nor right. This is why, it is known as CONTEXT - FREE. *Many programming languages have recursive structure that can be defined by CFG's.*

Example 1 : Consider the grammar $G = (V, T, P, S)$ having productions :
$S \rightarrow aSa \mid bSb \mid \in$. Check the productions and find the language generated.

Solution :

Let $P_1 : S \rightarrow aSa$ (RHS is terminal variable terminal)

$P_2 : S \rightarrow bSb$ (RHS is terminal variable terminal)

$P_3 : S \rightarrow \in$ (RHS is null string)

Since, all productions are of the form $A \rightarrow \alpha$, where $\alpha \in (V \cup T)^*$, hence G is a CFG.

2. Derivations Using a Grammar

Example 2 : Let G = (V, T, P, S) where V = { S , C }, T = { a , b }
P = { S → aCa
C → aCa | b
} S is the start symbol
What is the language generated by this grammar ?

Solution : Consider the derivation

$S \Rightarrow aCa \Rightarrow$ aba (By applying the 1^{st} and 3^{rd} production)

So, the string aba $\in L(G)$

Consider the derivation

$S \Rightarrow aCa$	By applying	$S \rightarrow aCa$	
$\Rightarrow aaCaa$	By applying	$C \rightarrow aCa$	
$\Rightarrow aaaCaaa$	By applying	$C \rightarrow aCa$	
.....			
.....			
$\Rightarrow a^nCa^n$	By applying	$C \rightarrow aCa$	n times
$\Rightarrow a^nba^n$	By applying	$C \rightarrow b$	

So, the language L accepted by the grammar G is $L(G) = \{ a^n\, ba^n \mid n \geq 1 \}$

i. e., the language L derived from the grammar G is "The string consisting of n number of a's followed by a 'b' followed by n number of a's.

3.Leftmost and rightmost derivation

5 . 2 LEFTMOST AND RIGHTMOST DERIVATIONS

Leftmost derivation :

If $G = (V, T, P, S)$ is a CFG and $w \in L(G)$ then a derivation $S \underset{L}{\overset{*}{\Rightarrow}} w$ is called leftmost derivation if and only if all steps involved in derivation have leftmost variable replacement only.

Rightmost derivation :

If $G = (V, T, P, S)$ is a CFG and $w \in L(G)$, then a derivation $S \underset{R}{\overset{*}{\Rightarrow}} w$ is called rightmost derivation if and only if all steps involved in derivation have rightmost variable replacement only.

Example 1 : Consider the grammar $S \rightarrow S + S \mid S * S \mid a \mid b$. Find leftmost and rightmost derivations for string $w = a * a + b$.

Solution :

Leftmost derivation for $w = a * a + b$

$S \underset{L}{\Rightarrow} S * S$ (Using $S \rightarrow S * S$)

$\underset{L}{\Rightarrow} a * S$ (The first left hand symbol is a, so using $S \rightarrow a$)

$\underset{L}{\Rightarrow} a * S + S$ (Using $S \rightarrow S + S$, in order to get $a + b$)

$\underset{L}{\Rightarrow} a * a + S$ (Second symbol from the left is a, so using $S \rightarrow a$)

$\underset{L}{\Rightarrow} a * a + b$ (The last symbol from the left is b, so using $S \rightarrow b$)

Rightmost derivation for $w = a * a + b$

$S \underset{R}{\Rightarrow} S * S$ (Using $S \rightarrow S * S$)

$\underset{R}{\Rightarrow} S * S + S$ (Since, in the above sentential form second symbol from the right is * so, we can not use $S \rightarrow a \mid b$. Therefore, we use $S \rightarrow S + S$)

$\underset{R}{\Rightarrow} S * S + b$ (Using $S \rightarrow b$)

$\underset{R}{\Rightarrow} S * a + b$ (Using $S \rightarrow a$)

$\underset{R}{\Rightarrow} a * a + b$ (Using $S \rightarrow a$)

4.The Language of grammar

5.Sequential form

6.Parse tree or derivation tree

Derivation

- The Production rules are used to derive certain strings. We will now formally define the language generated by grammar $G = (V, T, P, S)$. The generation of language using specific rules is called derivation.

Ex: Consider a language having any number of a's over $\Sigma = \{a\}$

Production rules

$S \rightarrow aS$ — rule 1

$S \rightarrow \epsilon$ — rule 2

- Now if Want "aaaaa" string to be derived we can start with start symbols

S
aS — rule 1
aaS — rule 1
aaaS — rule 1
aaaaS — rule 1
aaaaaS — rule 1
aaaaaϵ — rule 2
= aaaaa

Derivation Tree / Parse Tree

→ Root vertex — start symbol — Non-Terminal

→ leaf Vertex — Terminals

→ Intermediate Vertex — Non-Terminal

- if you consider any context Free Grammar is there any string accepted by CFG can be derived by using some derivation that can be graphically represented by using this derivation tree

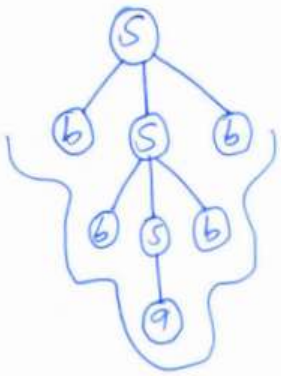
Ex: $S \rightarrow bSb \mid a \mid b$ construct the "bbabb" string

$S \rightarrow bSb \rightarrow bbSbb \rightarrow bbabb$

derivation

bbabb

Derivation Tree:



— Read the leaf nodes from left to right

bbabb

Types of Derivation Tree

- We have 2 types of Derivation Tree
  1. Left most Derivation Tree
  2. Right most Derivation Tree

Left most Derivation Tree :-

- Derivation will be done from left most non-terminal in production Rule

Right most Derivation Tree :-

- Derivation will be done from right most non-terminal in production Rule

Ex:-

$E \rightarrow E+E$
$E \rightarrow E*E$
$E \rightarrow a/b/c$

construct left most and right most derivation tree
string = a+b*c

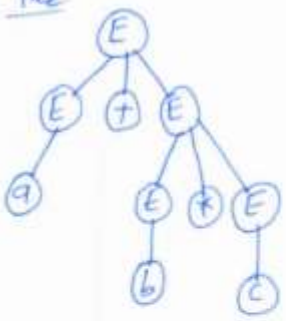
left most

E
↓
E+E
↓
a+E
↓
a+E*E
↓
a+b*c

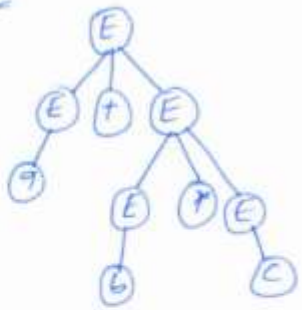
right most

E
↓
E+E
↓
E+E*E
↓
E+E*c
↓
E+b*c
↓
a+b*c

Tree



Tree



Ex:

$S \to TOOT$

$T \to OT$

$T \to IT$

$T \to \epsilon$

string = 1000111

Left most derivation

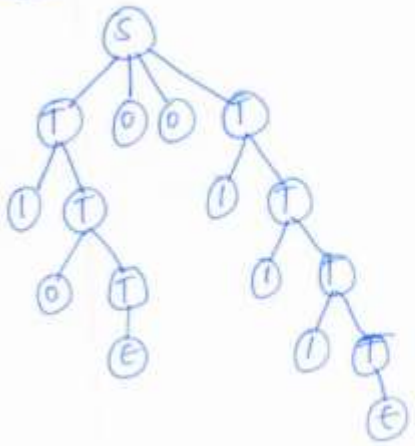
S
↓
TOOT
↓
ITOOT
↓
IOTOOT
↓
IOεOOT
IOOOT
↓
IOOOIT
↓
IOOOIIT
↓
IOOOIIIT
↓
IOOOIIIε
IOOOIII

Right most derivation

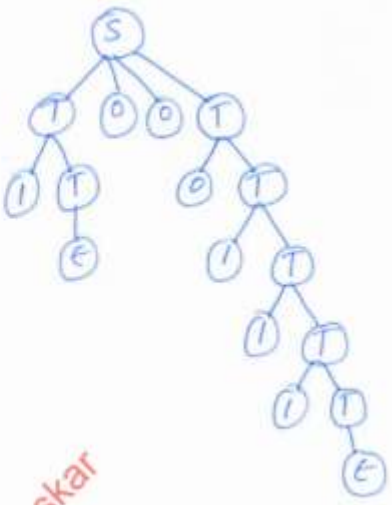
S
↓
TOOT
↓
TOOIT
↓
TOOIIT
↓

S
↓
TOOT
↓
TOOOT
↓
TOOOIT
↓
TOOOIIT
↓
TOOOIIIT
↓
TOOOIIIε
TOOOIII
↓
ITOOOIII
↓
IεOOOIII
IOOOIII

Tree



Tree



Ex:-

7.Ambriguity in grammar and language

Ambiguity in Grammar and Languages

— Ambiguous grammar is a grammar using which if it is possible to construct more than one Parse trees for the same string.

— if the string can be constructed by using more than one left most derivations (or) more than one right most derivations then such type of grammar known as Ambiguous grammar.

Ex: $S \rightarrow ABA$
$A \rightarrow aA \mid \epsilon$
$B \rightarrow bB \mid \epsilon$

string: aa

Left most derivation

S
↓
ABA
↓
aABA
↓
aaABA
↓
aaϵBA
aaBA
↓
aaϵA
aaA
↓
aaϵ
aa

ABA
↓
ϵBA
BA
↓
ϵA
A
↓
aA
↓
aaA
↓
aaϵ
aa

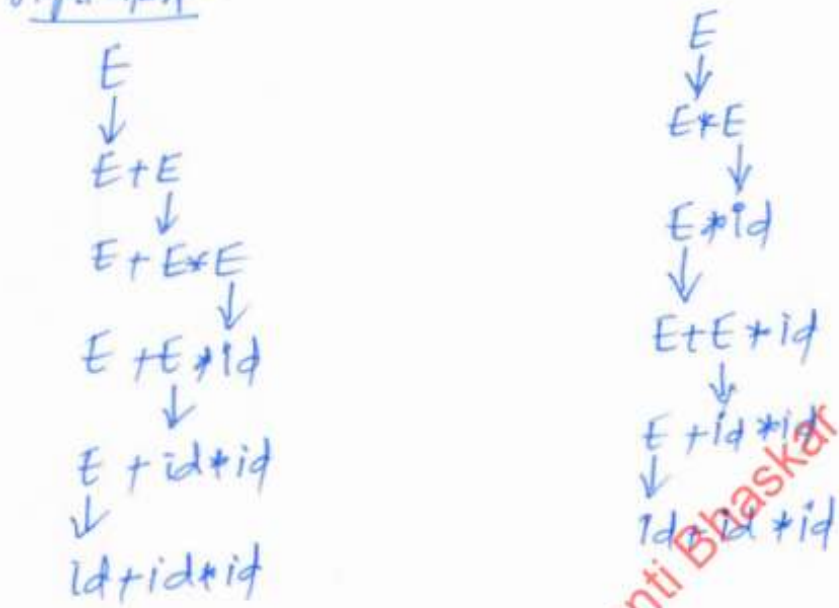
∴ So, the given grammar is ambiguous

This grammar is not suitable for construction of Compilers

In this case remove ambiguity in grammar

Ex: $E \to E+E$
$E \to E*E$
$E \to id$

String : id+id*id



- Find an unambiguous CFG equivalent to the grammar with Productions. $S \to aaaS/aaaaaaaS/\Lambda$
- When do you say a language L is unambiguous? Show that the language $L = \{a^n b^n / n >= 1\}$ is unambiguous
- Define ambiguous grammar. Check whether the grammar $S \to aAB$, $A \to bC/cd$, $C \to cd$, $B \to c/d$ is ambiguous (or) not.

8>Application of cfg

- **Compiler Design**: Context-free grammars (CFGs) define syntax rules for programming languages, enabling parsers to analyze and validate code structure during compilation.
- **Natural Language Processing**: CFGs model sentence structures in human languages, supporting tasks like parsing, sentiment analysis, and machine translation.
- **XML/HTML Parsing**: CFGs describe nested structures of markup languages, facilitating parsing and validation of web documents.
- **Query Language Processing**: CFGs define syntax for database query languages like SQL, allowing parsing and execution of complex queries.
- **Speech Recognition**: CFGs represent grammatical patterns in spoken language, aiding in the interpretation of voice inputs.
- **Protocol Specification**: CFGs model communication protocols, ensuring correct message formats in network systems.
- **Code Generation**: CFGs guide the generation of structured code or templates in software tools and IDEs.

PushDown Automata

1.

Push Down Automata

- Push Down Automata is a machine to recognise the context free grammars(CFG).
- CFG is a type-2 grammar which is recognised by PDA.
- R.G is a type-3 grammar which is recognised by FSM which Process finite Amount of information but PDA Process Infinite Amount of information. With the help of STACK by 2 operations (PUSH and POP).
- PDA contains 3 important components:
  1. Input Tape : input string
  2. Finite control : Push/Pop (every time the state can be changed to another state)
  3. stack : Elements
- Push Down Automata we can Rerpresent:

  PDA = FSM + STACK
  (5 tuple) (2 tuple)

- The PDA can be defined as a collection of 7 components:

$(Q, \Sigma, \delta, \Gamma, q_0, Z_0, F)$

Q: set of finite no. of states

Σ: Alphabet with i/p symbols

δ: Transition Function $\delta(q, a, x) \rightarrow \{(q_m, \alpha), \ldots\}$

$q \rightarrow$ Current state

$a \rightarrow$ i/p to be Processed

$x \rightarrow$ Top of stack

$q_m \rightarrow$ new state

$\alpha \rightarrow$ Top of stack to be replaced of x (string of stack symbols)

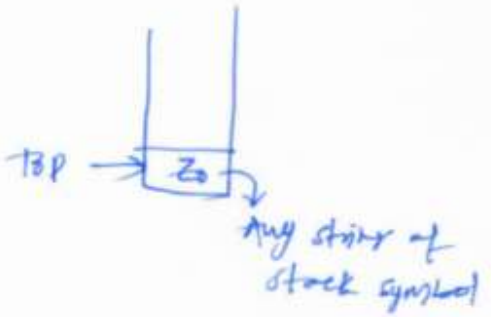
(14)

Γ : stack symbols

q_0 : Initial state

Z_0 : Initial top of stack

F : Final state



2.Language of pda

Languages of a PDA:-

- The defined languages $L(P)$ can be accepted by PDA.

Acceptance by Final state:-

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. Then the language $L(P)$ is called the language accepted by final state.

The language $L(P)$ can be defined as

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (P, \epsilon, \alpha)\}$$

i.e. The PDA P reads the entire input w and enters in a final state P.

Acceptance by Empty stack:-

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be the PDA then the language accepted by empty stack $N(P)$ is defined as

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (P, \epsilon, \epsilon)\}$$

i.e. starting from the initial configuration P reads the input w and empties its stack.

3. Deterministic Pushdown Automata

- Deterministic Pushdown Automata is a collection of
  1. Finite set of states Q.
  2. The input set Σ.
  3. Γ is a set of stack alphabets
  4. q_0 is the initial state. $q_0 \in Q$
  5. Z_0 is the start symbol which is in Γ
  6. Set of final states $F \subseteq Q$
  7. The δ is a mapping function used for moving from current state to next state, which can be as given below

$$\delta(q_0, X, Z_0) = \delta(q_1, X, \beta)$$

Where q_0 is the current state
X – current input symbol
Z_0 – current stack symbol
q_1 – next state
$X\beta$ shows top of the stack.

---

* If δ denotes unique transition for each input then the Pushdown automata is said to be deterministic Pushdown Automata.

Design a PDA for accepting a language $\{L = a^n b^n / n \geq 1\}$

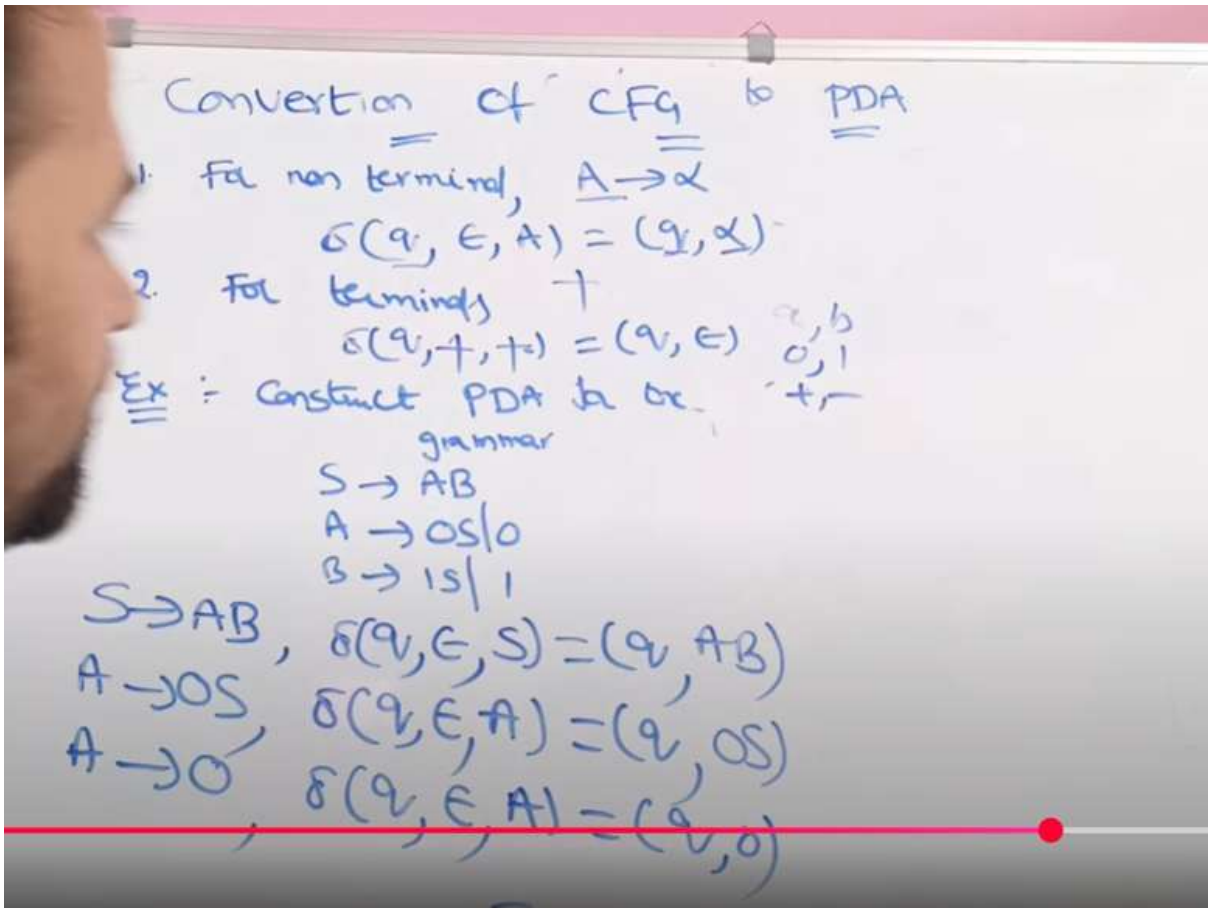
$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$
$$\delta(q_0, a, a) = (q_0, aa)$$
$$\delta(q_0, b, a) = (q_1, \epsilon)$$
$$\delta(q_1, b, a) = (q_1, \epsilon)$$
$$\delta(q_1, \epsilon, Z_0) = (q_2, \epsilon)$$

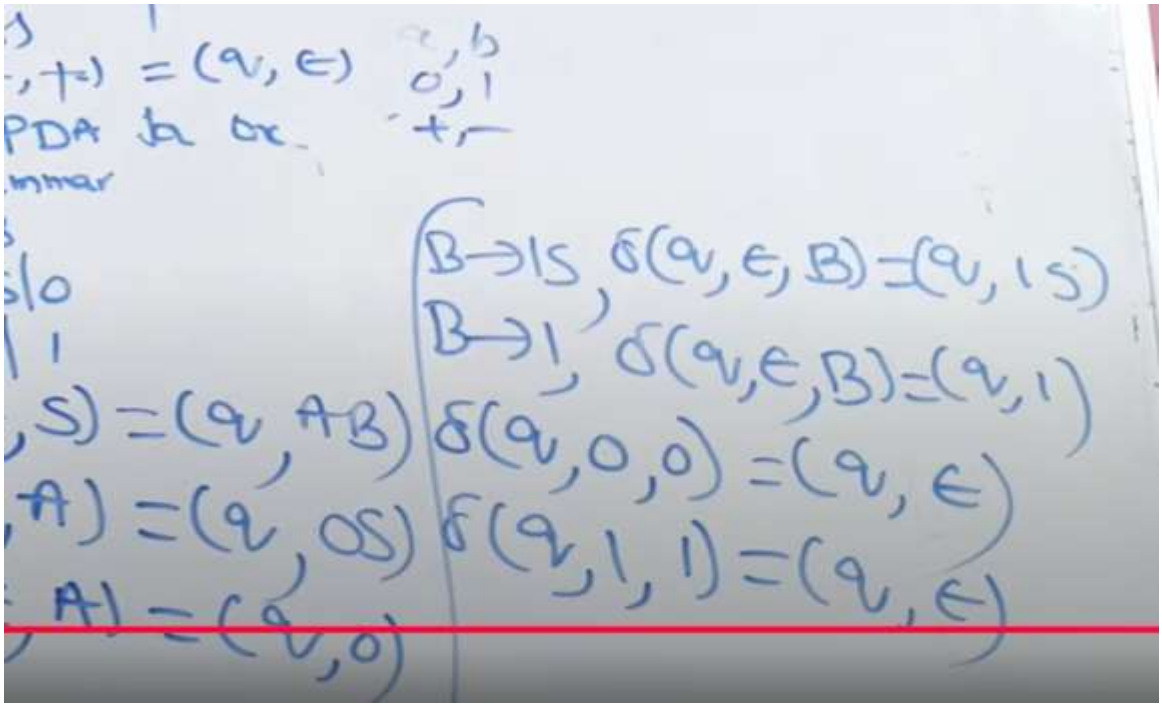
$\therefore$ transitions (δ) are unique, then the PDA is deterministic

---

4.convert cfg to pda

Mostly here we need to find transition function





5.PDA to CFG